

Diskussion

Vi har nu gått igenom grunderna i hur operatorer kan överlagras och sett på fördelarna man kan uppnå. Så länge som man använder överlagring av operatorer på ett vettigt sätt blir klasser mycket lättare att använda, och följdaktligen program lättare att läsa. Man har mycket att vinna genom korrekt hantering av överlagring. I fel händer är det dock frågan om ett formidabelt vapen som kan åstadkomma mycket oreda och problem i program. Speciellt om man försöker ändra på de ursprungliga operatorernas betydelse.

Det finns många olika operatorer som vi inte ens tittat på, men som för många klasser är vettiga, t.ex. operatorerna `++` och `□`, men de är relativt lätta att implementera med hjälp av exemplen i detta kapitel.

Ett bra exempel på klasser där överlagring av operatorer använts i mycket stor skala är de olika klasserna i *Standard Template Library* (se [Kapitel 24](#)). Där överlagras en stor mängd olika operatorer, såsom `=`, `[]`, `++`, `□` samt många andra. Utan överlagrade operatorer skulle program som använder sig av STL vara ganska svårlästa, men nu är komplexiteten delvis gömd bakom ett enhetligt gränssnitt som uppnåtts genom noggrann överlagring av operatorer.

Den kompletta klassen för `Vector` finns i [Appendix C](#).